

Lecture 03

Lists and Tuples

OCaml Carefully So Far

- *A program is a sequence of bindings*
- Typecheck each binding in order
 - Using **static environment** from previous bindings
- Evaluate each binding in order
 - Using **dynamic environment** from previous bindings
- So far: No way to make *compound data* (like Java objects, arrays, ...)
 - That's important, so let's get to it!

Compound Data Types

- So far: integers, booleans, variables, conditionals, functions
- Now: ways to build data with multiple parts
 - **Tuples**: fixed number of “pieces” with possibly different types
 - **Lists**: any number of “pieces” all with the same type
- Later: more general ways to create compound data
 - trees, maps, graphs, etc.

General Design Principle

More systematic approach
to defining / understanding
programming languages!

Whenever we design a new type **t** we need two things:

1. A way to **build** (“introduce”, “construct”) values of type **t**
2. A way to **use** (“eliminate”, “destruct”) values of type **t**

General Design Principle

More systematic approach
to defining / understanding
programming languages!

Whenever we design a new type t we need two things:

1. A way to **build** (“introduce”, “construct”) values of type t
2. A way to **use** (“eliminate”, “destruct”) values of type t

Example:

Build $t_1 * t_2 * \dots * t_n \rightarrow t$ with function bindings, use with function calls

General Design Principle

More systematic approach
to defining / understanding
programming languages!

Whenever we design a new type t we need two things:

1. A way to **build** (“introduce”, “construct”) values of type t
2. A way to **use** (“eliminate”, “destruct”) values of type t

Example:

Build $t_1 * t_2 * \dots * t_n \rightarrow t$ with function bindings, use with function calls

Example:

Build **int** with 0, 1 etc, use with $1 + 7$, $5 * 4$, etc



Pairs (2-tuples) : Build

Syntax: (e_1, e_2) where e_1 and e_2 are expressions

Type Checking

- If e_1 has type t_1 and e_2 has type t_2
- Then (e_1, e_2) has type $t_1 * t_2$
- Else, report error and fail (only if e_1 or e_2 does not type-check)

Evaluation

- If e_1 evaluates to value v_1 and e_2 evaluates to value v_2
- Then (e_1, e_2) evaluates to (v_1, v_2)

The Star of the Show

- `*` is a type constructor: it builds tuple types out of other types
 - Ex: `int * bool`, `string * int`, `int * (int * int)`
 - Infinite number of types can be constructed with `*`
- Tuple types are also known as product types
- But the connection to multiplication is obscure and best ignored
 - Just using the same character in unrelated syntax
 - `3 * 4` is an expression that evaluates to 12, `int * int` is a type

Nested Pairs

Pairs can be nested

- Nesting is **NOT** a new feature!
- Simply a consequence of the syntax and semantics rules we set up

`((1,4),(2,(true,3)) : (int * int) * (int * (bool * int)))`

- Compositionality is a hallmark of good design
 - Often from orthogonality

Pairs (2-tuples) : Use

Syntax: `fst e` and `snd e`

- Where `e` is an expression

Type Checking

- If `e` has type `t1 * t2`, then
- `fst e` has type `t1`
- `snd e` has type `t2`

Evaluation

- If `e` evaluates to a pair of values `(v1, v2)`, then
- `fst e` evaluates to `v1`
- `snd e` evaluates to `v2`

Fib alert: `fst` and `snd` are actually library functions, but we will pretend they are “built in” for another week or so.

Tuples : Build

Syntax: $(e_1, \dots, e_N)$

- Where $e_1, \dots, e_N$ are expressions and $N \geq 2$
- **New feature:** $(5, \text{true}, 7)$ is a 3-tuple (triple), *not* a nested pair

Type Checking

- If e_1 has type t_1 and ... and e_N has type t_N
- Then $(e_1, \dots, e_N)$ has type $t_1 * \dots * t_N$
- Else, report error and fail

Evaluation

- If e_1 evaluates to value v_1 and ... and e_N evaluates to value v_N
- Then $(e_1, \dots, e_N)$ evaluates to $(v_1, \dots, v_N)$

Tuples : Use

- OCaml does not have a built-in way to use arbitrary tuples...
- Instead, OCaml has a more general mechanism we will learn soon
 - Pattern-matching
- But Homework 1 uses triples a lot
 - So we wrote `fst3`, `snd3`, and `thd3` for you to treat like primitives
 - So `fst` and `snd` for pairs; `fst3`, `snd3`, and `thd3` for triples
 - Why can't `fst` be used for triples? Type systems have limitations

Tuples : Use

- OCaml does not have a built-in way to use arbitrary tuples...
- Instance soon
 - Can we write one function, `fstN`, that returns the first element out of *any* tuple?
Why or why not?
- But primitives
 - So `fst` and `snd` for pairs; `fst3`, `snd3`, and `thd3` for triples
 - Why can't `fst` be used for triples? Type systems have limitations

Nested pairs vs. tuples

- OCaml didn't *need* tuples, but they can be convenient.
- Parentheses matters in tuple types and tuple expressions!

```
let x = (5,7,9)
x : int * int * int
let seven = snd3 x
snd x doesn't type-check
```

```
let x = (5,(7,9))
x : int * (int * int)
let seven = fst (snd x)
let pr     = snd x
snd3 x doesn't type-check
```

Compound Data Types

- So far: integers, booleans, variables, conditionals, functions
- Now: ways to build data with multiple parts
- Plan
 - **Tuples**: fixed number of “pieces” with possibly different types
 - **Lists**: any number of “pieces” all with the same type
- Later: more general ways to create compound data
 - trees, maps, graphs, etc.

Lists : Build

Syntax:

- V

Type Che

- If
- T
- E

Evaluation

- If
- T

New kind of type!

t list

Another **type constructor** (**list**) for making the type:
lists whose elements have type **t**.

Examples:

```
bool list    int list
(int * (int * int)) list
int list list
```

lue **vN**

Lists : Build

Syntax: $[e_1; \dots; e_N]$

- Where $e_1, \dots, e_N$ are expressions

Type Checking

- If e_1 has type t and $\dots$ and e_N has type t
- Then $[e_1; \dots; e_N]$ has type t list
- Else, report error and fail

Evaluation

- If e_1 evaluates to value v_1 and $\dots$ and e_N evaluates to value v_N
- Then $[e_1; \dots; e_N]$ evaluates to $[v_1; \dots; v_N]$

Lists : Build

Elements separated by semicolons “ ; ” ⚠
Get very weird error messages otherwise ⚠

Syntax: [e1; ...; eN]

- Where e1, ..., eN are expressions

Type Checking

- If e1 has type t and ... and eN has type t
- Then [e1; ...; eN] has type t list
- Else, report error and fail

All same type t

Evaluation

- If e1 evaluates to value v1 and ... and eN evaluates to value vN
- Then [e1; ...; eN] evaluates to [v1; ...; vN]

Lists of values are values.

Lists : Build

Syntax: `[e1; ...; eN]`

- Where `e1`, ..., `eN` are expressions

Type Checking

- If `e1` has type `t` and ... and `eN` has type `t`
- Then `[e1; ...; eN]` has type `t` list
- Else, report error and fail

Evaluation

- If `e1` evaluates to value `v1` and ... and `eN` evaluates to value `vN`
- Then `[e1; ...; eN]` evaluates to `[v1; ...; vN]`

`N` can be zero!

`[]` is the empty list for **any** type.

[]

- If `e1` has type `t` and ... and `eN` has type `t`
- Then `[e1; ...; eN]` has type `t list`

So if **all zero elements of [] have type `t`**, then `[]` has type `t list`

- `[]` has type `int list`, `bool list`, `int list list`, `(bool*int) list`, ...
- OCaml writes, “any type of list” as `'a list` (or `'b list`, `'c list`, ...)
 - But all list elements of any list have the same type
- Will see these *polymorphic types* a lot in OCaml

Lists : Build with “cons” (::)

“Create (construct) a list with one more element at the front”

Syntax: $e1 :: e2$

- Where $e1$ and $e2$ are expressions

Type Checking

- If $e1$ has type t and $e2$ has type t list
- Then $e1 :: e2$ has type t list
- Else, report error and fail

Evaluation

- If $e1$ evaluates to $v1$ and $e2$ evaluates to $[v2; \dots; vN]$
- Then $e1 :: e2$ evaluates to $[v1; v2; \dots; vN]$

Two ways to build?

- `::` is more common in code
- `::` and `[]` is all we *need*
 - `e1::e2::e3::...::en::[]` is the same as `[e1;e2;e3;...;en]`
 - First of many examples of “**syntactic sugar**”
- List values are printed as `[v1;v2;v3;...;vn]`

Lists : Use

- For now, we will *use* lists with these features:
 - Test if a list is empty with `e = []`
 - Get the *first* element of a *non-empty list* with function `List.hd`
 - Get the *rest* of a *non-empty list* with function `List.tl`
 - (a list with everything after the first element)
- `List.hd` and `List.tl` will raise an exception if given an empty list
- Later, will use *pattern matching* to do this more safely and elegantly

Lists : Use -- Test for empty

Syntax: `e = []`

- Where `e` is an expression

Type Checking

- If `e` has type `t` list
- Then `e = []` has type `bool`
- Else report error and fail

Evaluation

- If `e` evaluates to value `[]`, then `e = []` evaluates to `true`
- Else `e = []` evaluates to `false`

Lists : Use -- Get the first element

Syntax: `List.hd e`

- Where `e` is an expression

Type Checking

- If `e` has type `t list`
- Then `List.hd e` has type `t`
- Else, report error and fail

Alternate definition:

`List.hd` has type `'a list -> 'a`

Evaluation

- If `e` evaluates to value `[v1; v2; ...; vN]`
- Then `List.hd e` evaluates to value `v1`
- Else `e` evaluated to `[]`, so `List.hd e` raises an exception

Lists : Use -- Get the rest (everything after first)

Syntax: `List.tl e`

- Where `e` is an expression

Type Checking

- If `e` has type `t list`
- Then `List.tl e` has type `t list`
- Else, report error and fail

Alternate definition:

`List.tl` has type `'a list -> 'a list`

Evaluation

- If `e` evaluates to value `[v1; v2; ...; vN]`
- Then `List.tl e` evaluates to value `[v2; ...; vN]`
- Else `e` evaluated to `[]`, so `List.tl e` raises an exception

Recursion again!

- Functions over lists are usually recursive
 - Only way to “get to all the elements”
- Design recipe: answer two questions
 - *What should the answer be for an empty list?*
 - Base case
 - Often thinking about the return type provides a good hint
 - *What should the answer be for a non-empty list?*
 - Recursive case
 - Typically this will be in terms of the answer for the tail

Lists of tuples

Especially on HW1!!



- Processing lists of tuples is a common pattern
 - Requires *no new features!*
 - “Just” compose what we’ve already learned
 - Again, compositionality is a hallmark of good design
 - often from orthogonality

Group Work

Write the following OCaml functions:

1. `sum_pair` that sums the integers in an `int * int` pair
 - Example: `sum_pair (1, 3) = 4`
2. `sum_list` that sums the elements in an `int list`
 - Example: `sum_list [1; 2; 3] = 6`
3. `sum_pair_list` that sums the elements in an `(int * int) list`
 - Example: `sum_pair_list [(1, 2); (3, 4)] = 10`
4. `sum_list_pair` that sums the elements in an `(int list) * (int list)` pair
 - Example: `sum_list_pair ([1; 2; 3], [4; 5; 6]) = 21`